Online Learning Applications Project

Mohammad Javad Zandiyeh (11044962)

June 26, 2026

Contents

1 Introduction **3**

2 Problem Formulation **3**

2.1 Single-campaign setting . . . 3

2.2 Multi-campaign setting . . . 3

3 Implementation Overview **3**

3.1 Repository structure . . . 3

3.2 Central design choices . . . 4

3.3 Single-campaign optimization oracle . . . 5

3.4 Multi-campaign optimization oracle . . . 5

4 Datasets and Experimental Protocol **5**

4.1 Synthetic scenarios . . . 5

4.2 Evaluation protocol . . . 6

5 Requirement 1: Single-Campaign Stochastic Bidding **6**

5.1 Implemented algorithms . . . 6

5.2 Results summary . . . 6

5.3 Interpretation . . . 8

6 Requirement 2: Multi-Campaign Stochastic Bidding **8**

6.1 Implemented algorithm . . . 8

6.2 Results summary . . . 9

6.3 Interpretation . . . 10

7 Requirement 3: Best-of-Both-Worlds Primal-Dual Learning **10**

7.1 Implemented algorithm . . . 10

7.2 Results summary . . . 11

7.3 Interpretation . . . 13

8 Requirement 4: Piecewise-Stationary Learning **13**

8.1 Implemented algorithms 13
8.2 Results summary 14
8.3 Interpretation 15

9 Cross-Cutting Discussion of Unexpected Results **15**

10 Conclusion **16**

A Complete Plot Gallery **16**
A.1 Requirement 1: all plots 16
A.2 Requirement 2: unique plots 16
A.3 Requirement 3: all plots 16
A.4 Requirement 4: all plots 16

1 Introduction

This report documents the implementation and empirical evaluation of an online learning project for budget-constrained bidding in repeated first-price auctions. The repository implements four progressively harder requirements:

- Requirement 1: single-campaign stochastic bidding;
- Requirement 2: multi-campaign stochastic bidding with a global budget and conflict constraints;
- Requirement 3: a best-of-both-worlds primal-dual method that works in both stochastic and highly non-stationary environments;
- Requirement 4: piecewise-stationary bidding with sliding-window and change-detection extensions of Combinatorial-UCB.

The goal of the project is not only to run the algorithms, but to implement the full pipeline: synthetic scenario generation, clairvoyant baselines, budget-safe simulation, reproducible experiments, and diagnostic plots that explain both successful and unexpected behaviors.

2 Problem Formulation

2.1 Single-campaign setting

At each round $t = 1, \dots, T$, the learner chooses a bid b_t from a discrete set $\mathcal{B} \subset [0, 1]$. The environment draws the highest competing bid m_t, and the learner wins iff $b_t \geq m_t$. In a first-price auction,

$$f_t(b_t) = (v - b_t)\mathbf{1}\{b_t \geq m_t\}, \qquad c_t(b_t) = b_t\mathbf{1}\{b_t \geq m_t\},$$

where v is the campaign valuation. The learner has a finite budget B, so the online policy must trade off utility maximization against pacing the spend over time.

2.2 Multi-campaign setting

In the multi-campaign setting there are N campaigns, one bid can be selected per campaign, and all campaigns share a *single global budget*. Two extra constraints appear:

- some campaigns cannot be activated together because of a conflict graph;
- feedback may be semi-bandit or full-information depending on the requirement.

The per-round budget target is denoted by $\rho = B/T$. The natural clairvoyant benchmarks therefore optimize expected or empirical utility under the long-term constraint $\mathbb{E}[c_t] \leq \rho$.

3 Implementation Overview

3.1 Repository structure

The implementation is organized as follows:

```
src/ola/
  algorithms/
    ucb1.py
    ucb_budget.py
    combinatorial_ucb.py
    primal_dual.py
    nonstationary_ucb.py
    pacing.py
  auction.py
  baseline.py
  conflict_graph.py
  distributions.py
  environment.py
  metrics.py
  simulation.py

data/
  generate_datasets.py
  scenarios/*.json
  samples/*.csv

experiments/
  run_requirement1.py
  run_requirement2.py
  run_requirement3.py
  run_requirement4.py
```

The code uses only a small scientific Python stack: `numpy`, `scipy`, `matplotlib`, and `networkx`. This keeps the project lightweight while still supporting LP solving, random sampling, graph operations, and figure generation.

3.2 Central design choices

Several implementation choices are shared by all requirements:

- **Centralized budget bookkeeping.** Budget enforcement is implemented in `simulation.py`, not inside the bidders. This guarantees that all learners are evaluated under the same stopping rule.
- **Budget-safe termination.** In the single-campaign case the learner stops as soon as the remaining budget is below the maximum bid. In the multi-campaign case it stops when the remaining budget cannot cover the worst-case joint action, namely

$$(\text{max independent set size}) \times (\text{max bid}).$$

- **Explicit baselines.** All regret curves are measured against clairvoyant benchmarks implemented in `baseline.py`, either from the true underlying distributions or from the realized non-stationary sequence.
- **Reproducibility.** The dataset generator writes JSON scenario files and reproducible sample CSVs. The experiment scripts fix the random seeds and write summary CSV files used by this report.

3.3 Single-campaign optimization oracle

For Requirement 1, the constrained bidding LP

$$\max_{\gamma \in \Delta(\mathcal{B})} \sum_b \gamma(b)\, \bar{f}(b) \quad \text{s.t.} \quad \sum_b \gamma(b)\, \bar{c}(b) \le \rho$$

is solved in closed form in `baseline.py`. Because the feasible region has only the simplex constraint and one budget constraint, an optimal vertex is supported on at most two bids. The implementation therefore enumerates feasible pure bids and cost-straddling pairs rather than calling a generic LP solver. This is both efficient and mathematically transparent.

3.4 Multi-campaign optimization oracle

For Requirements 2–4, the per-round oracle solves the combinatorial LP

$$\max_x \sum_{i,b} f_i(b) x_{i,b}$$

subject to:

- at most one bid per campaign;
- total expected cost at most ρ;
- conflict constraints induced by the graph;
- non-negativity.

This oracle is implemented with `scipy.optimize.linprog`. The conflict graphs used in the project are matchings (disjoint pairs), so the rounding scheme in `combinatorial_ucb.py` is exact for the intended setting: free campaigns are Bernoulli-rounded independently, while conflict pairs are rounded mutually exclusively.

4 Datasets and Experimental Protocol

4.1 Synthetic scenarios

All experiments use a common bid grid

$$\mathcal{B} = \{0, 0.05, 0.10, \ldots, 1.00\},$$

with horizon $T = 10{,}000$. The dataset generator produces:

- five single-campaign distributions: uniform, low competition, high competition, beta-skewed, and bimodal;
- three stationary multi-campaign scenarios: independent, conflict, and correlated;
- four non-stationary scenarios for Requirement 3: `ns_abrupt`, `ns_smooth`, `ns_fast`, `ns_conflict`;
- two dedicated piecewise-stationary scenarios for Requirement 4: `pw_slight` and `pw_frequent`, plus the reused abrupt and conflict cases.

The budget regimes are deliberately designed to stress the algorithms:

- **Binding budget:** ρ is set to half of the unconstrained expected or hindsight spend, so the constraint is genuinely active.
- **Slack budget:** the budget is large enough that the constraint is effectively inactive.

4.2 Evaluation protocol

The experiment scripts use the following protocol:

- Requirement 1: 5 seeds;
- Requirement 2: 5 seeds;
- Requirement 3: 5 seeds for Primal-Dual and 3 seeds for Combinatorial-UCB;
- Requirement 4: 4 seeds, plus a smaller 2-seed window-sensitivity sweep.

The main outputs are regret curves, budget curves, diagnostic plots, and compact CSV summaries. In the non-stationary setting, the report distinguishes between:

- a **fixed hindsight optimum**, which is the correct benchmark for a best-of-both-worlds guarantee;
- a **dynamic optimum**, which can adapt to changes and is more appropriate for piecewise-stationary tracking.

5 Requirement 1: Single-Campaign Stochastic Bidding

5.1 Implemented algorithms

UCB1 baseline. The file `ucb1.py` implements standard UCB1 over the discrete bid set. It uses realized auction utility as the arm reward and ignores the budget during arm selection. This is a useful baseline because it is a valid no-regret method when the budget is slack, but it has no built-in pacing mechanism.

Budgeted UCB. The file `ucb_budget.py` implements a bandits-with-knapsacks style learner. For each bid it maintains:

$$f^{\mathrm{UCB}}(b) = \hat{f}(b) + \mathrm{radius}_f(b), \qquad c^{\mathrm{LCB}}(b) = \hat{c}(b) - \mathrm{radius}_c(b),$$

then solves the constrained LP using optimistic utility and pessimistic cost. Unseen bids are assigned maximal utility optimism and zero cost pessimism, which forces exploration.

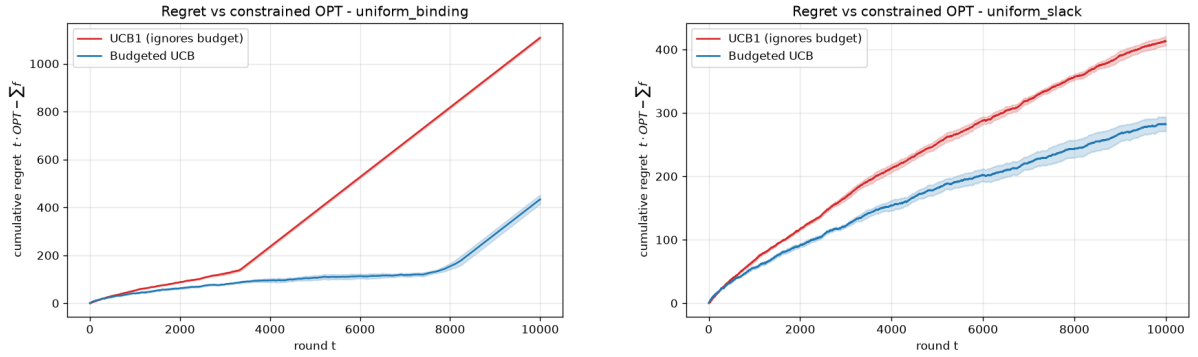
5.2 Results summary

Scenario	ρ	UCB1 util	UCB1 stop	Budgeted util	$T \cdot OPT$
`uniform_binding`	0.0800	346.5	3325	1020.9	1454.5
`low_competition_binding`	0.1630	1631.3	4812	3076.8	3672.3
`high_competition_binding`	0.2596	360.7	8854	336.9	862.4
`beta_skewed_binding`	0.1533	1109.3	4678	2153.5	2778.7
`bimodal_binding`	0.0831	633.5	3140	1473.2	2286.0

Table 1: Requirement 1 headline results on the binding-budget scenarios.

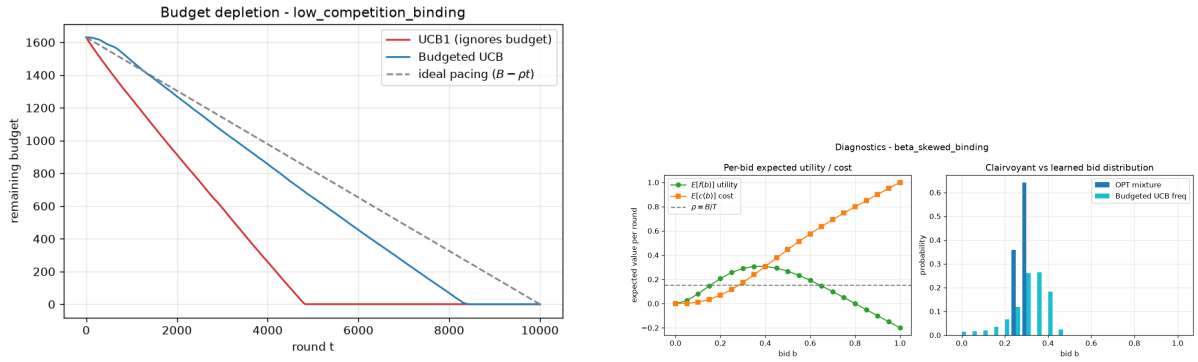
The main pattern is clear: when the budget matters, ignoring it is expensive. UCB1 tends to learn the unconstrained best bid, overspends early, and then earns zero utility for the rest of the horizon. Budgeted UCB, instead, tracks the per-round budget target and survives much longer.

The only notable exception is `high_competition_binding`, where the budgeted learner slightly underperforms UCB1. This is an important result rather than a failure to hide: when competition is strong, profitable wins are rare and expensive, so the cautious cost pessimism of Budgeted UCB can make it overly conservative in finite horizon. This is a good example of the remark that unexpected results deserve discussion.



(a) `uniform_binding` (b) `uniform_slack`

Figure 1: Representative Requirement 1 regret curves. Under a binding budget, UCB1 eventually depletes the budget and its regret becomes linear; under a slack budget, both learners behave like valid no-regret stochastic bandits.



(a) Budget pacing (b) Learned bid distribution

Figure 2: Requirement 1 diagnostics: the budget-aware learner paces the spend and learns a bid distribution close to the constrained clairvoyant optimum.

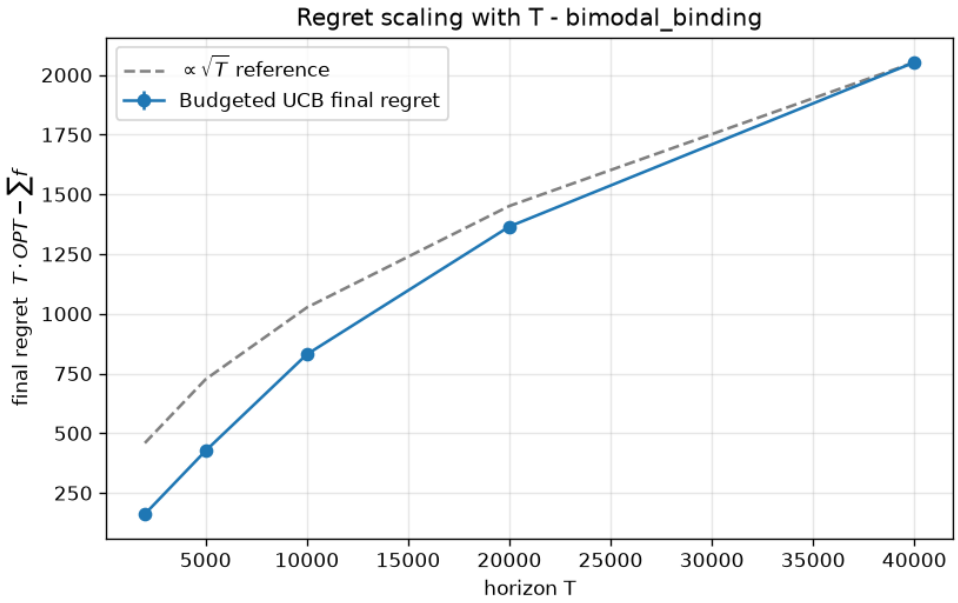


Figure 3: Requirement 1 scaling experiment. The final regret of Budgeted UCB grows approximately like $\sqrt{T}$, which is consistent with the expected sublinear regret behavior.

5.3 Interpretation

Requirement 1 validates the basic methodology of the project. The gap between the two learners appears only when the constraint is active, exactly as theory suggests. The additional diagnostic plots are especially useful because they show that the improvement is not accidental: the budget-aware learner is not just spending less, it is learning the correct *mixture of bids*.

6 Requirement 2: Multi-Campaign Stochastic Bidding

6.1 Implemented algorithm

`combinatorial_ucb.py` extends Requirement 1 to multiple simultaneous campaigns. Each pair (i, b) is treated as an arm, the confidence bounds are computed per arm, and a linear optimization oracle chooses the best feasible superarm under:

- one bid per campaign,
- a global budget constraint,
- conflict-graph constraints.

The learner receives semi-bandit feedback, so only played campaign-bid pairs are updated.

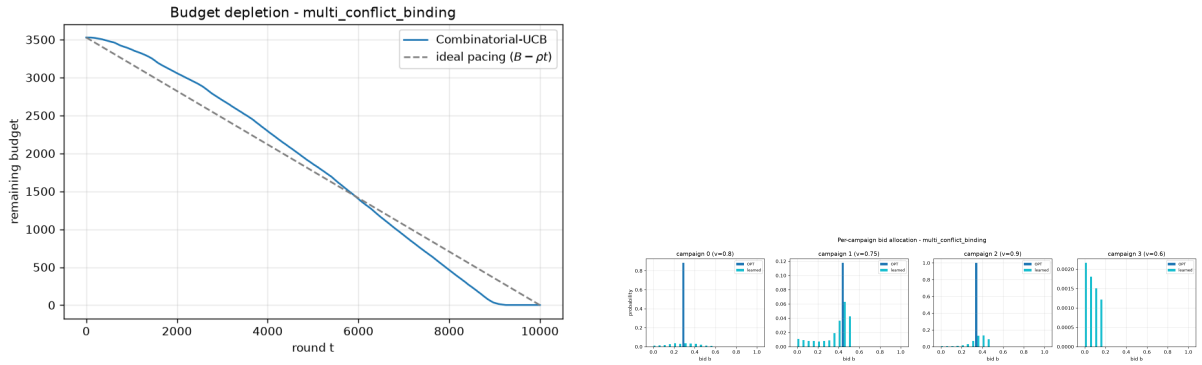
6.2 Results summary

Scenario	N	corr	ρ	Comb-UCB util	$T \cdot OPT$
`multi_independent_binding`	3	0.0	0.5027	5652.8	6350.1
`multi_conflict_binding`	4	0.0	0.3527	4315.6	5302.0
`multi_correlated_binding`	3	0.6	0.3164	4567.6	6713.1

Table 2: Requirement 2 results on the binding-budget scenarios.

The algorithm performs well across all three stationary multi-campaign regimes. The summary CSV shows that the realized utility stays reasonably close to the clairvoyant benchmark, the budget is respected, and the diagnostics show that the empirical bid allocation concentrates on the oracle allocation.

The conflict case is particularly important because it validates the rounding logic: the algorithm learns to drop the lower-value campaign in each conflict pair, which is exactly the qualitative behavior expected from the oracle.



(a) Budget pacing

(b) Conflict-aware allocation

Figure 4: Requirement 2 diagnostics. The learner both respects the global budget and recovers the correct campaign-level allocation in the presence of conflicts.

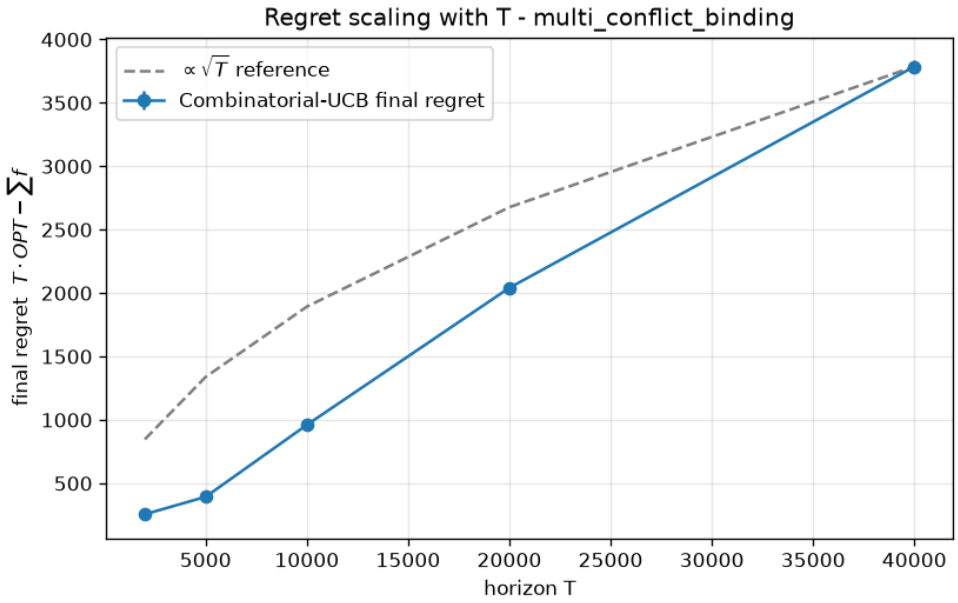


Figure 5: Requirement 2 scaling experiment. The final regret again follows an approximately $\sqrt{T}$ trend.

Note that the final repository plot bundle reuses the filenames `regret_multi_*` inside Requirement 3 for the direct Primal-Dual vs Combinatorial-UCB comparison. For that reason, the unique Requirement 2 visual evidence in this report is the budget, diagnostic, and scaling plots, while the stationary regret numbers are taken from `summary_req2.csv`.

6.3 Interpretation

Requirement 2 shows that the project is not merely a single-bandit simulation. The implementation successfully combines budgeted exploration, graph-constrained feasibility, and joint action rounding. Correlation makes the environment harder, but the structure of the benchmark is still recovered because expected utility and cost depend only on the marginal bid distributions.

7 Requirement 3: Best-of-Both-Worlds Primal-Dual Learning

7.1 Implemented algorithm

The key contribution of Requirement 3 is `PrimalDualBidder`. It combines:

- a single dual variable λ_t that prices the shared budget;
- a primal full-information learner based on Hedge;
- a factorization of the feasible action space into independent blocks induced by the matching conflict graph.

Given λ_t, each bid receives Lagrangian reward

$$L_{i,t}(b) = f_{i,t}(b) - \lambda_t c_{i,t}(b).$$

After observing the full vector of competing bids, the algorithm can score every counterfactual

action and update Hedge on each block. The dual variable is then updated via projected online gradient descent:

$$\lambda_{t+1} = \Pi_{[0,1/\rho]}\left(\lambda_t + \eta_{\text{dual}}(\widehat{c}_t - \rho)\right).$$

This design is precisely what makes the algorithm work in both stochastic and non-stationary environments: unlike UCB, Hedge never collapses exploration, and the budget is handled by the dual signal rather than by confidence bounds alone.

7.2 Results summary

Scenario	World	$T{\cdot}OPT_{\text{fixed}}$	$T{\cdot}OPT_{\text{dyn}}$	PD regret	CUCB regret
`ns_abrupt_binding`	non-stationary	5677.3	6283.0	689.9	222.1
`ns_smooth_binding`	non-stationary	5327.9	5375.9	1455.8	1909.7
`ns_fast_binding`	non-stationary	5510.7	5844.2	511.5	1172.9
`ns_conflict_binding`	non-stationary	3588.7	4667.2	438.9	522.4
`multi_independent_binding`	stochastic	6350.1	6350.1	430.8	699.7
`multi_conflict_binding`	stochastic	5302.0	5302.0	405.2	1024.2
`multi_correlated_binding`	stochastic	6713.1	6713.1	734.1	2153.6

Table 3: Requirement 3 comparison against the best fixed feasible policy in hindsight. The dynamic optimum is reported to quantify the price of non-stationarity.

This is the strongest section of the project. The same primal-dual learner is competitive in both worlds, while Combinatorial-UCB degrades badly whenever its stationarity assumptions become stale. On `ns_fast_binding`, for instance, the primal-dual regret is less than half the UCB-style regret.

At the same time, the results are nuanced rather than uniformly one-sided: `ns_abrupt_binding` is a case where Combinatorial-UCB still beats the primal-dual method with respect to the *fixed* benchmark. This is worth highlighting: best-of-both-worlds does not mean domination on every instance, only robustness across regimes. The project therefore correctly surfaces the trade-off instead of overselling the algorithm.

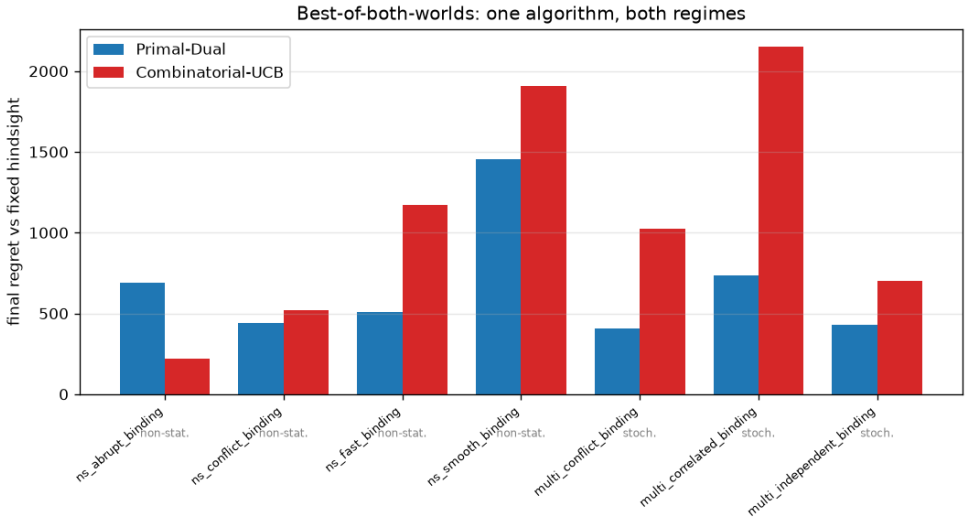
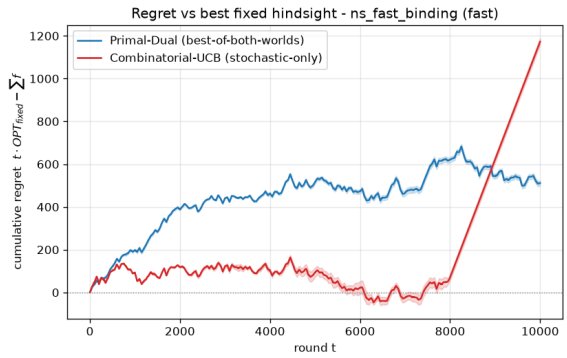
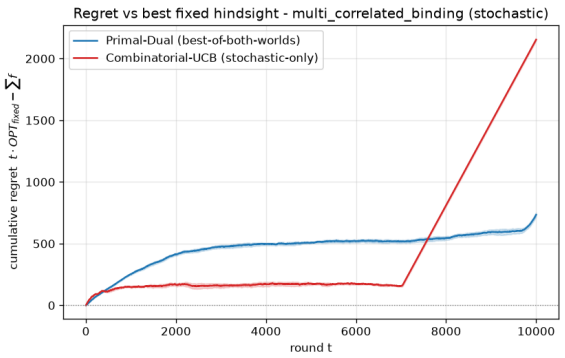


Figure 6: Requirement 3 summary plot. The primal-dual algorithm is consistently strong across stochastic and non-stationary regimes, while the stochastic-only baseline becomes unreliable when the environment changes.

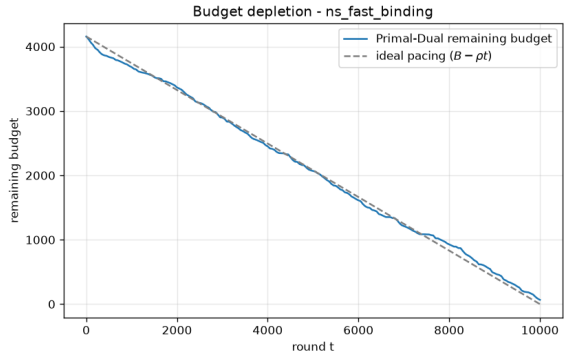


(a) `ns_fast_binding`

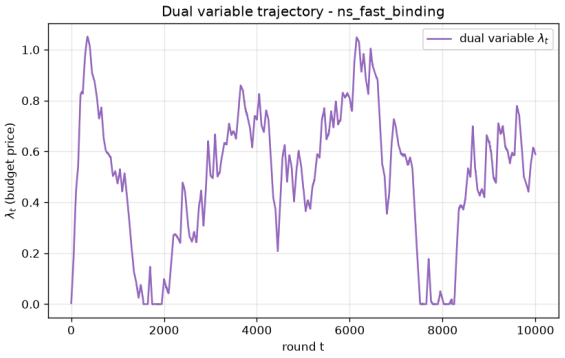


(b) `multi_correlated_binding`

Figure 7: Representative Requirement 3 regret curves. The primal-dual learner remains competitive both in a highly non-stationary regime and in the reused stationary stochastic regime.



(a) Budget pacing



(b) Dual trajectory

Figure 8: Requirement 3 diagnostics on `ns_fast_binding`. The dual variable acts as a real-time budget price, allowing the learner to react to changing competition while pacing the spend until the horizon.

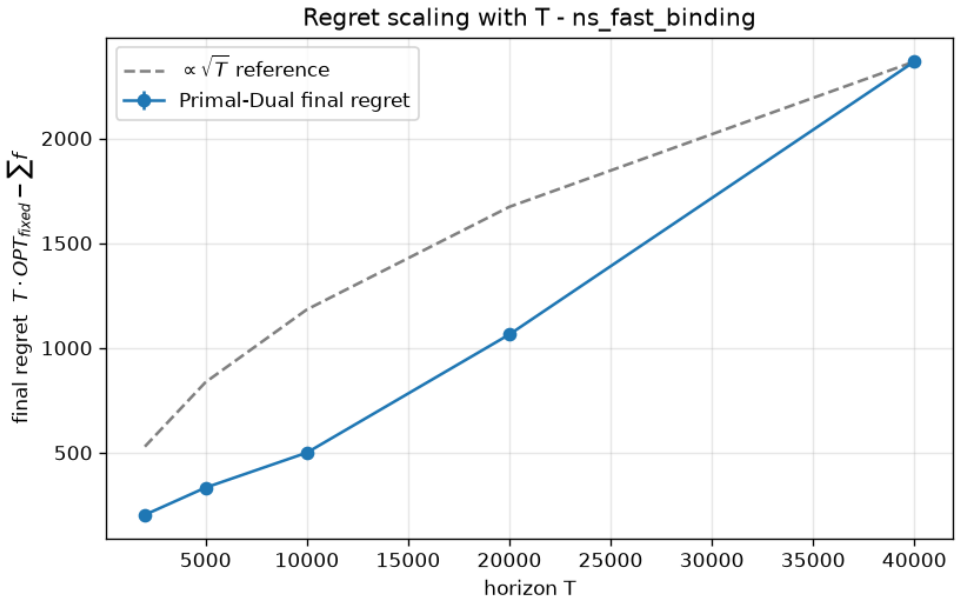


Figure 9: Requirement 3 scaling experiment on the fast non-stationary scenario. The final regret remains compatible with a sublinear trend.

7.3 Interpretation

The most convincing evidence for the primal-dual design is the *shape* of the curves. When Combinatorial-UCB fails, it usually fails in the same way: confidence bounds become stale, the algorithm overspends early, and once the budget is exhausted the regret turns into an almost linear ramp. The primal-dual learner avoids this failure mode because it keeps updating on full feedback and uses the dual variable to explicitly control pacing.

8 Requirement 4: Piecewise-Stationary Learning

8.1 Implemented algorithms

Requirement 4 extends Combinatorial-UCB in two classic ways:

- **SlidingWindowCombinatorialUCBBidder:** only the last W observations of each arm are kept. The default implementation uses $W \approx 20\sqrt{T}$, larger than the textbook $\sqrt{T}$ rule because the combinatorial setting spreads samples over many arms.
- **ChangeDetectionCombinatorialUCBBidder:** each arm is equipped with a CUSUM detector with parameters `cusum_m = 50`, `cusum_eps = 0.15`, `cusum_h = 5.0`, and exploration probability `alpha = 0.01`.

These are compared against the original Comb-UCB and the Requirement 3 Primal-Dual method.

8.2 Results summary

Scenario	$T \cdot OPT_{\text{dyn}}$	Comb-UCB	SW-CombUCB	CD-CombUCB	Primal-Dual
`pw_slight_binding`	6822.9	1622.2	**943.8**	1805.2	1050.7
`ns_conflict_binding`	4659.6	1576.5	1553.3	1688.4	**1513.3**
`ns_abrupt_binding`	6267.2	**867.5**	1350.9	1070.9	1281.1
`pw_frequent_binding`	5413.0	**748.2**	784.4	1489.4	985.7

Table 4: Requirement 4 final regrets against the dynamic per-interval optimum.

This requirement is interesting precisely because there is no universal winner. The results clearly show the price of robustness:

- SW-CombUCB wins when changes are relatively infrequent and the budget is tight (`pw_slight_binding`);
- Primal-Dual is strongest in the conflict-heavy budget-sensitive case (`ns_conflict_binding`);
- plain Comb-UCB is still hard to beat when the environment is not too far from stationary (`ns_abrupt_binding`, `pw_frequent_binding`);
- CD-CombUCB is the most sensitive method and performs poorly when resets and forced exploration consume too much of the budget.

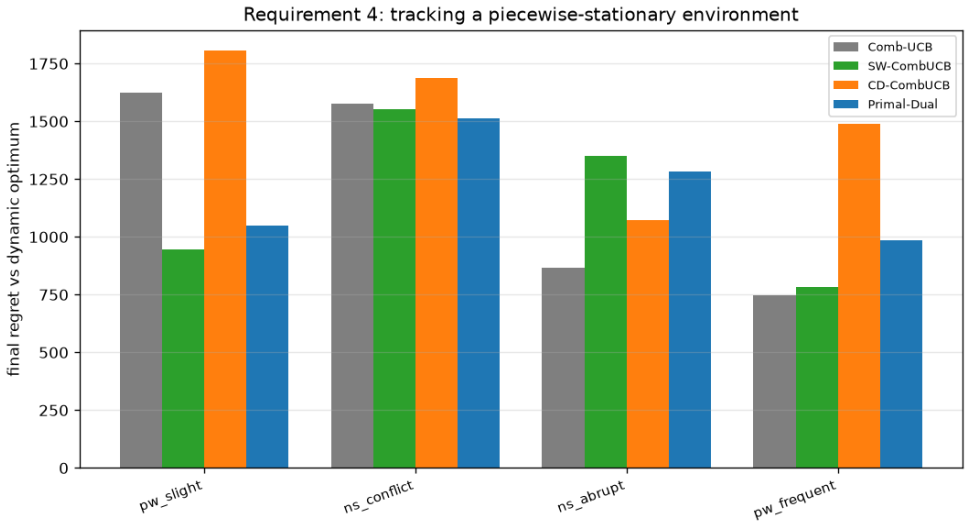
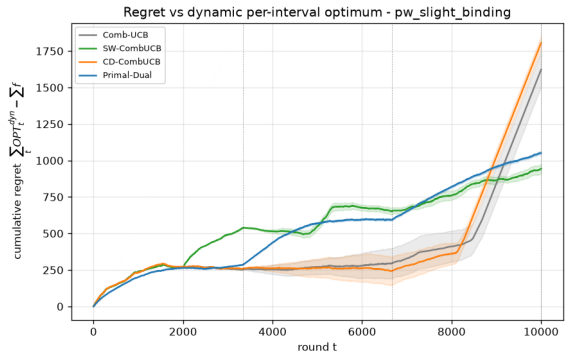
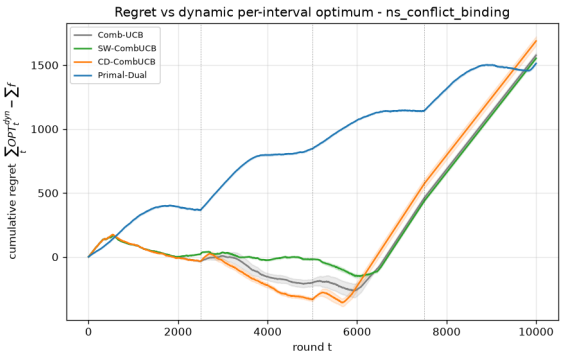


Figure 10: Requirement 4 summary comparison. The best method depends on the frequency of changes and on how expensive budget mistakes are.

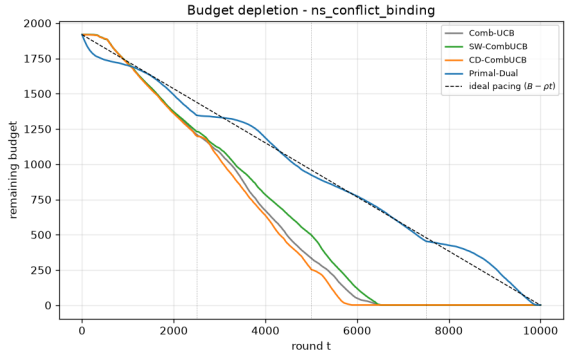


(a) `pw_slight_binding`

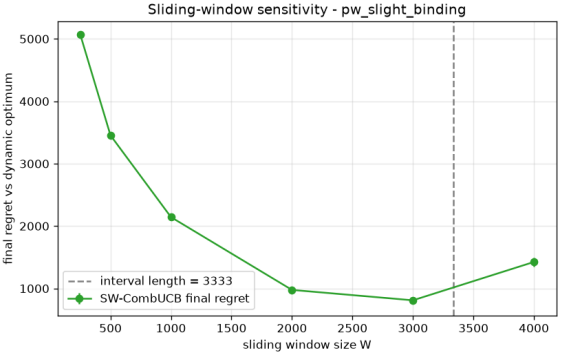


(b) `ns_conflict_binding`

Figure 11: Requirement 4 regret curves against the dynamic benchmark.



(a) Budget trajectories



(b) Sensitivity to the window size

Figure 12: Requirement 4 diagnostics. The conflict scenario highlights the value of primal-dual pacing, while the window sweep shows that the best W depends on the unknown interval length.

8.3 Interpretation

The project correctly captures a subtle point from the course: non-stationary methods can pay an unnecessary price when the world is close to stationary. This is why Comb-UCB remains competitive in some piecewise-stationary instances. The report should therefore not frame Requirement 4 as "newer algorithms always win"; instead, the correct conclusion is that adaptation mechanisms help only when their bias-variance trade-off matches the change frequency.

9 Cross-Cutting Discussion of Unexpected Results

Several findings deserve explicit discussion:

1. **Budget-aware learning is not always better in finite horizon.** The `high_competition_binding` scenario in Requirement 1 shows that stronger budget discipline can be too conservative when profitable wins are already scarce.

2. **Best-of-both-worlds does not imply pointwise dominance.** In Requirement 3, the primal-dual method is the most robust overall, but it does not beat Combinatorial-UCB on every single instance. The `ns_abrupt_binding` scenario is the clearest counterexample.

3. **Dynamic and fixed benchmarks tell different stories.** The large gap between $T \cdot OPT_{\text{fixed}}$ and $T \cdot OPT_{\text{dyn}}$ on scenarios such as `ns_conflict_binding` quantifies how much value is lost by insisting on a fixed action distribution in a changing environment.

4. **Change detection is fragile under budget constraints.** Resets and forced exploration are expensive when the budget is binding. This is why CD-CombUCB is often the weakest method in Requirement 4 despite being more adaptive in principle.

These are useful points for the oral presentation because they show a real understanding of the algorithms, not just a list of plots.

10 Conclusion

The project delivers a coherent implementation of budget-constrained bidding across stochastic, combinatorial, and non-stationary regimes. The strongest engineering aspects are:

- a clean separation between learning logic, simulation logic, and clairvoyant baselines;
- scenario generators that produce interpretable and reproducible instances;
- experiment scripts that are self-contained and produce both headline summaries and detailed diagnostics;
- careful comparison against the right benchmark for each requirement.

Empirically, the report supports four main conclusions:

- budget-aware learning matters as soon as the budget truly binds;
- combinatorial structure and conflicts can be handled effectively through an LP oracle and structured rounding;
- the primal-dual approach is the most robust method across stochastic and rapidly changing environments;
- no non-stationary method dominates universally, and the degree of non-stationarity strongly affects which algorithm is preferable.

A Complete Plot Gallery

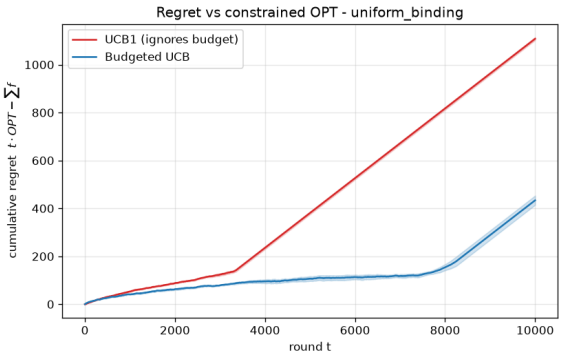
This appendix includes every generated figure copied into the report directory. The main body focused on the most informative plots; the appendix preserves the full experimental record.

A.1 Requirement 1: all plots

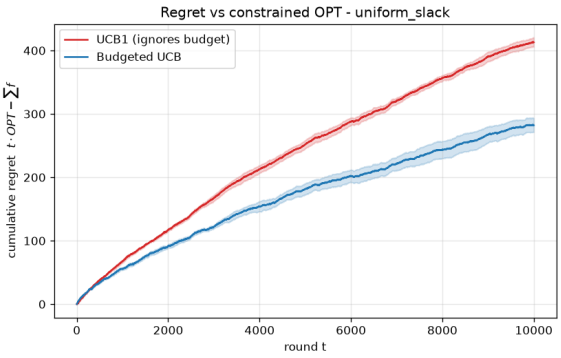
A.2 Requirement 2: unique plots

A.3 Requirement 3: all plots

A.4 Requirement 4: all plots

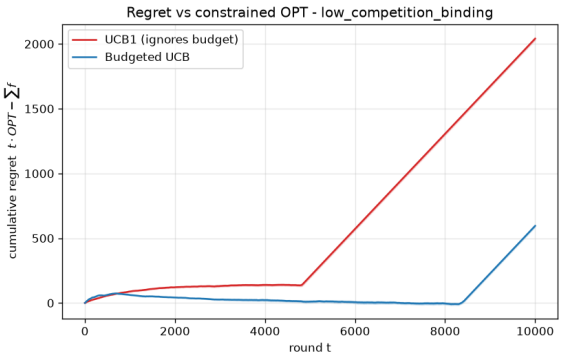


(a) `uniform_binding`

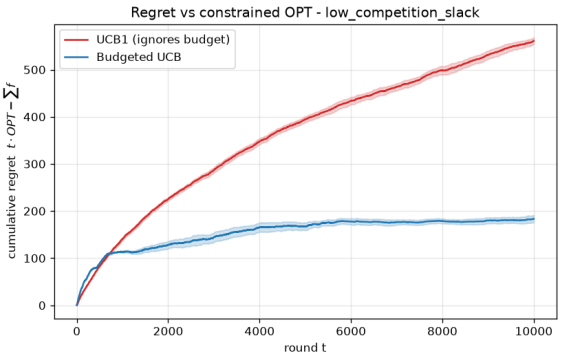


(b) `uniform_slack`

Figure 13: Requirement 1 regret plots: uniform scenario.



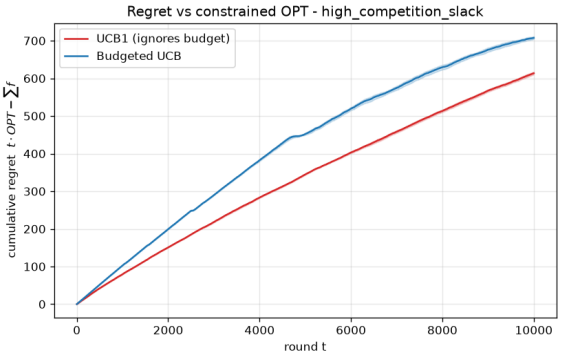
(a) `low_competition_binding`



(b) `low_competition_slack`

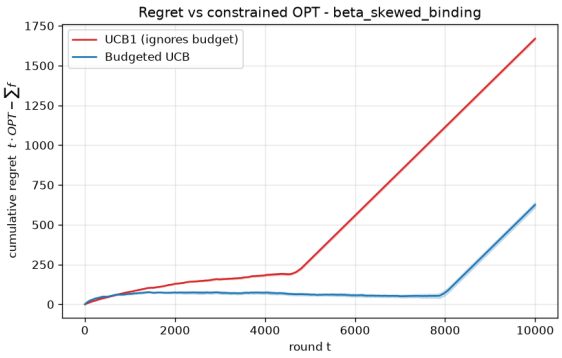
Figure 14: Requirement 1 regret plots: low-competition scenario.

(a) `high_competition_binding`

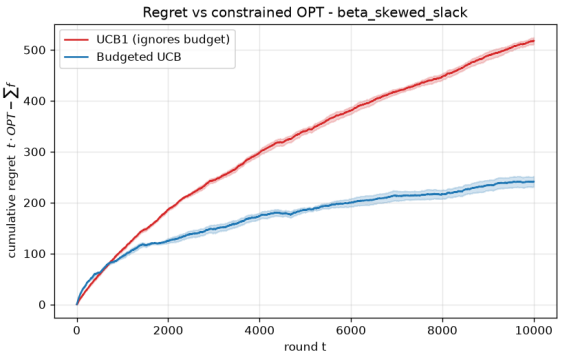


(b) `high_competition_slack`

Figure 15: Requirement 1 regret plots: high-competition scenario.

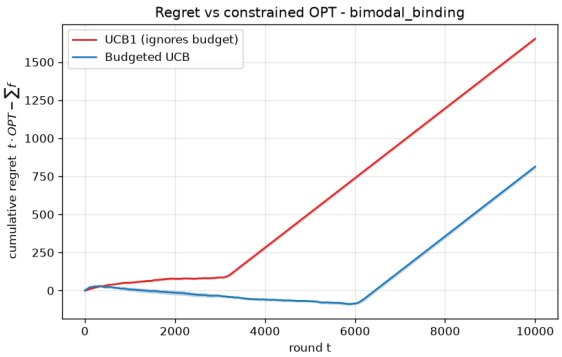


(a) `beta_skewed_binding`

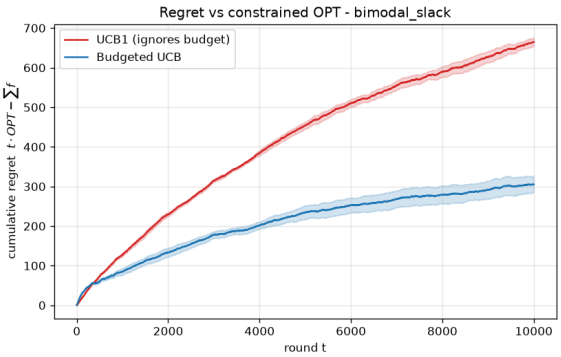


(b) `beta_skewed_slack`

Figure 16: Requirement 1 regret plots: beta-skewed scenario.

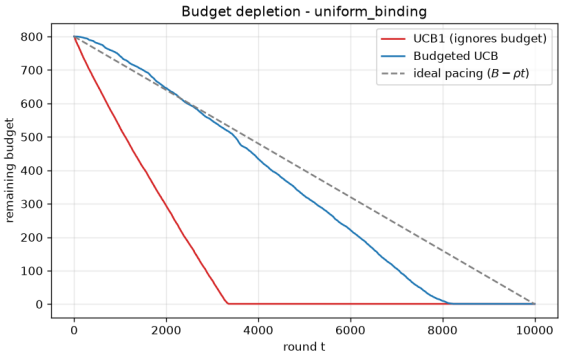


(a) `bimodal_binding`

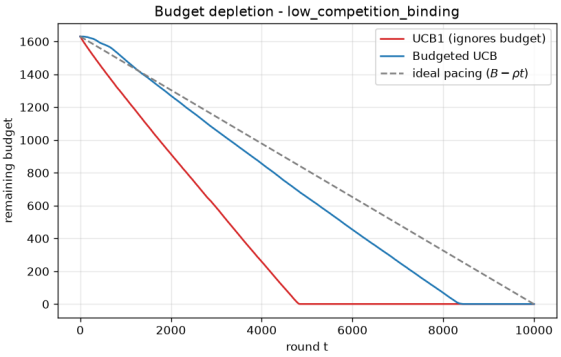


(b) `bimodal_slack`

Figure 17: Requirement 1 regret plots: bimodal scenario.

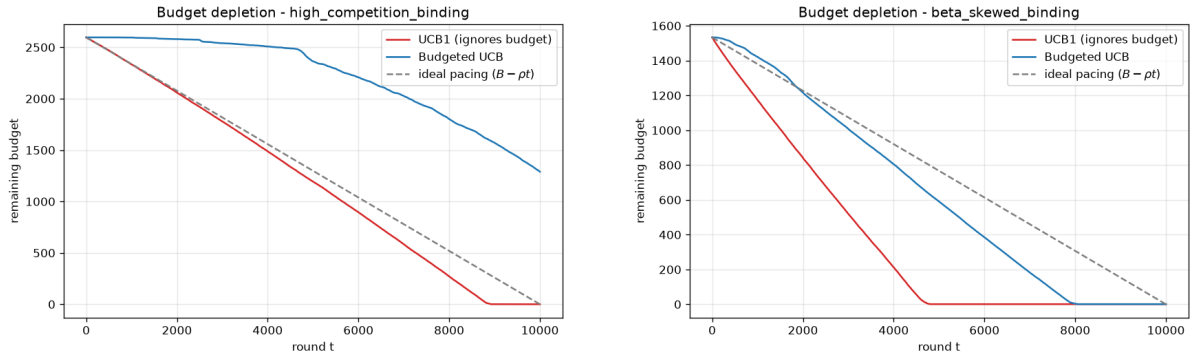


(a) `uniform_binding`



(b) `low_competition_binding`

Figure 18: Requirement 1 budget plots, part I.



(a) `high_competition_binding` (b) `beta_skewed_binding`

Figure 19: Requirement 1 budget plots, part II.

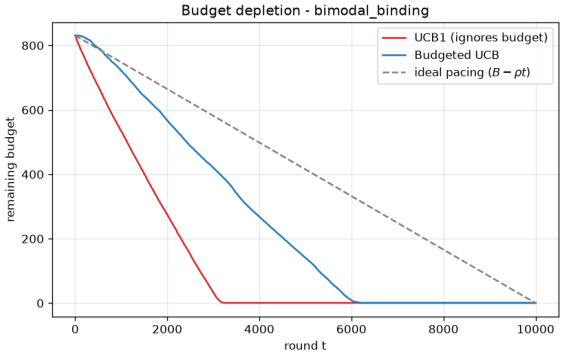
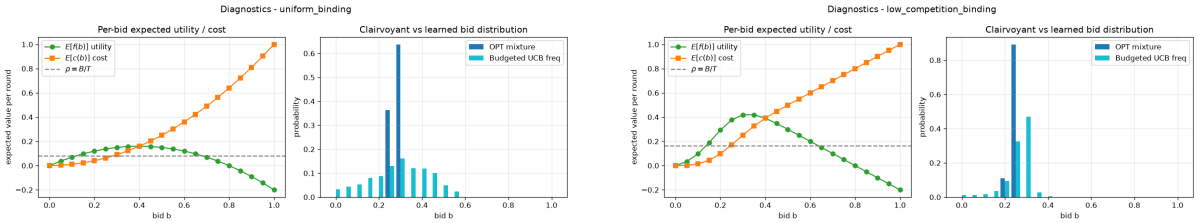
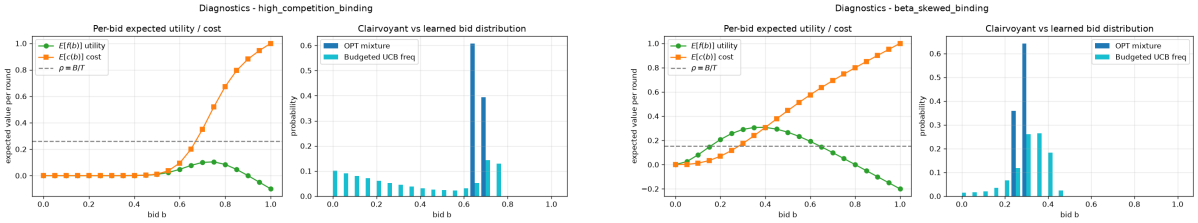


Figure 20: Requirement 1 budget plot: `bimodal_binding`.



(a) `uniform_binding` (b) `low_competition_binding`

Figure 21: Requirement 1 diagnostic plots, part I.



(a) `high_competition_binding` (b) `beta_skewed_binding`

Figure 22: Requirement 1 diagnostic plots, part II.

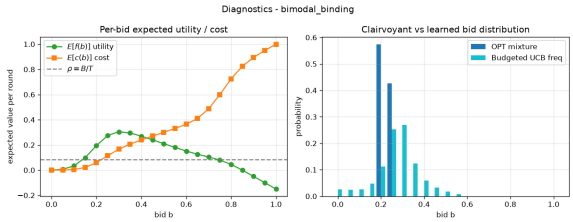


Figure 23: Requirement 1 diagnostic plot: `bimodal_binding`.

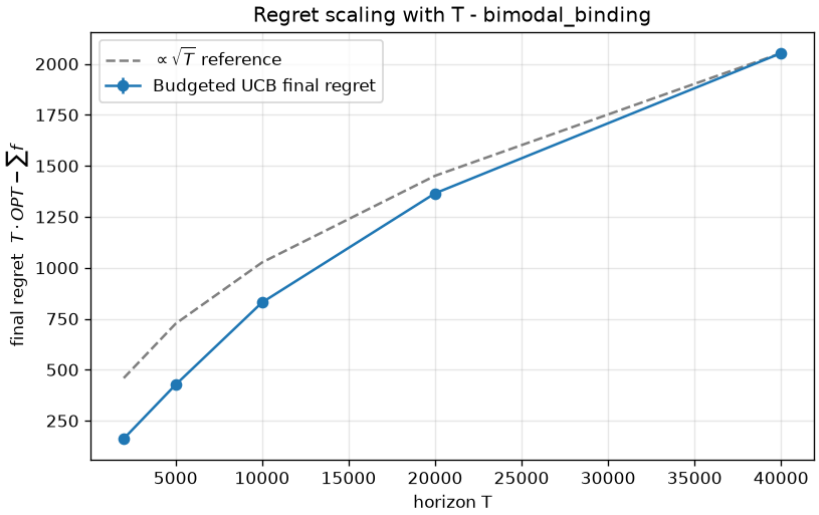
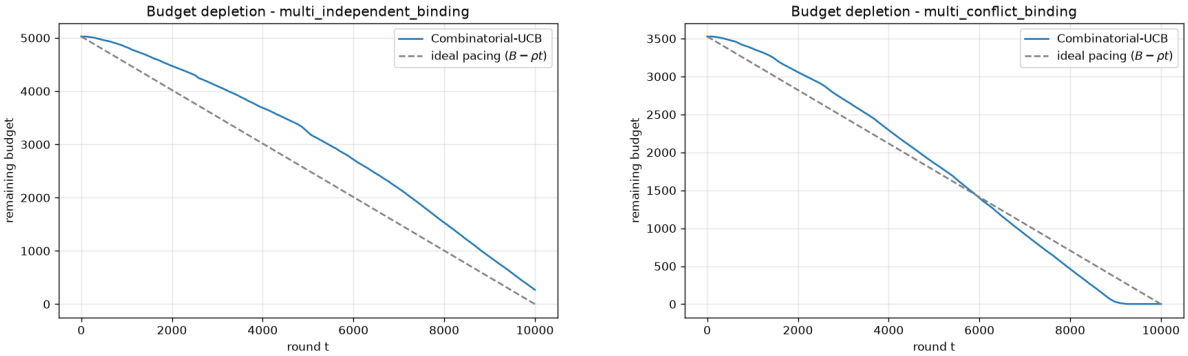


Figure 24: Requirement 1 scaling plot.



(a) `multi_independent_binding` (b) `multi_conflict_binding`

Figure 25: Requirement 2 budget plots, part I.

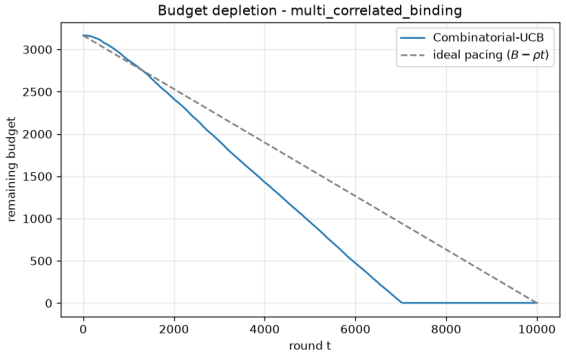
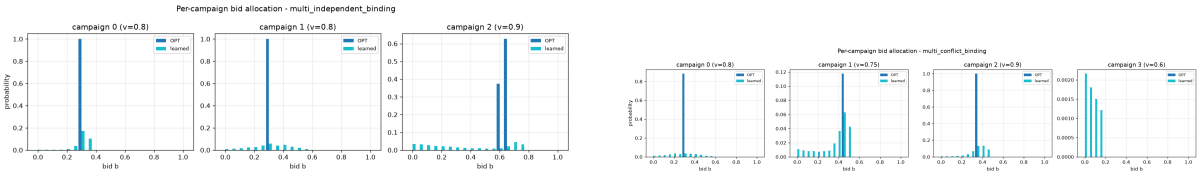


Figure 26: Requirement 2 budget plot: `multi_correlated_binding`.



(a) `multi_independent_binding` (b) `multi_conflict_binding`

Figure 27: Requirement 2 diagnostics, part I.

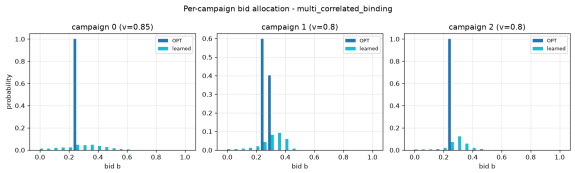


Figure 28: Requirement 2 diagnostic plot: `multi_correlated_binding`.

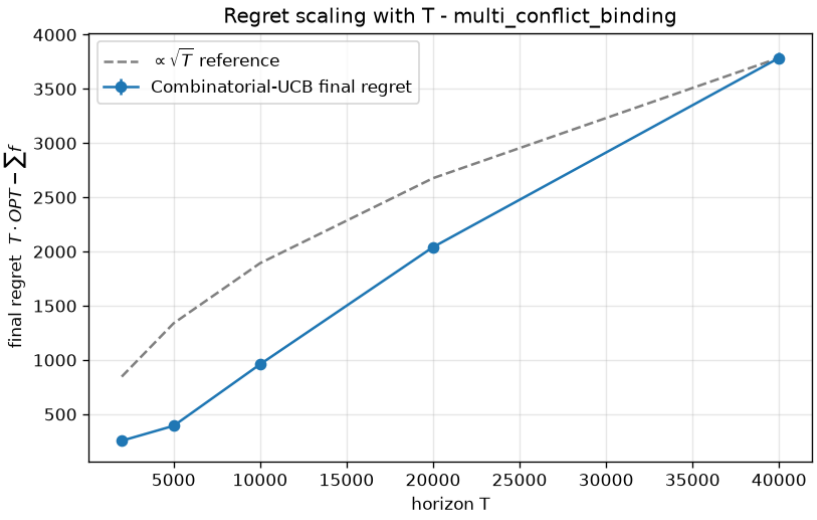
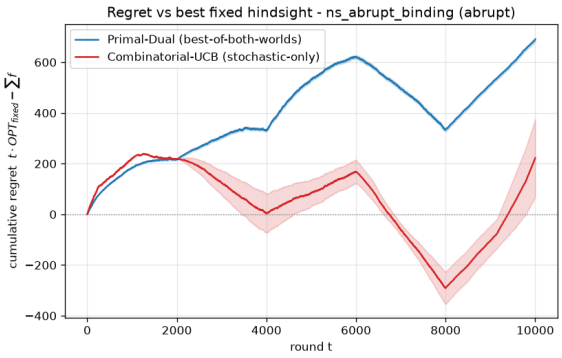
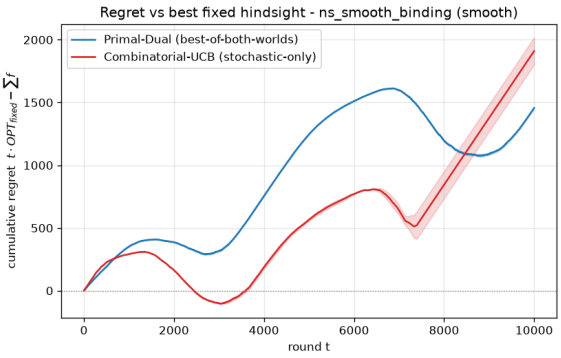


Figure 29: Requirement 2 scaling plot.

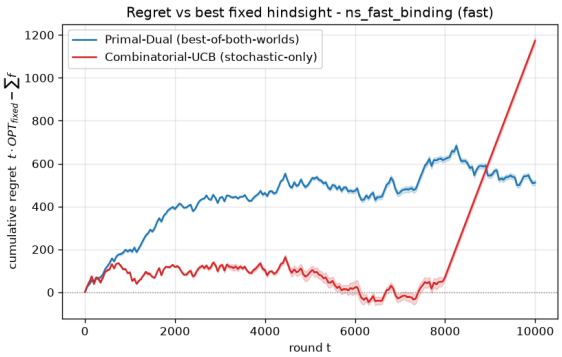


(a) `ns_abrupt_binding`

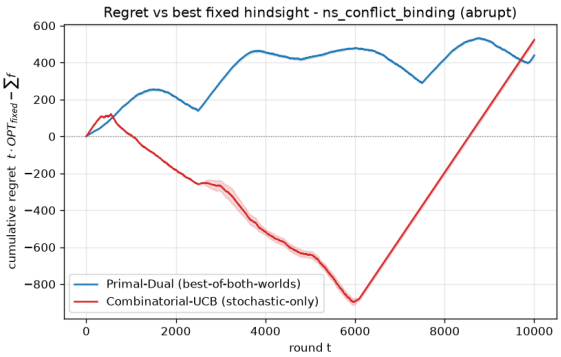


(b) `ns_smooth_binding`

Figure 30: Requirement 3 regret plots, part I.

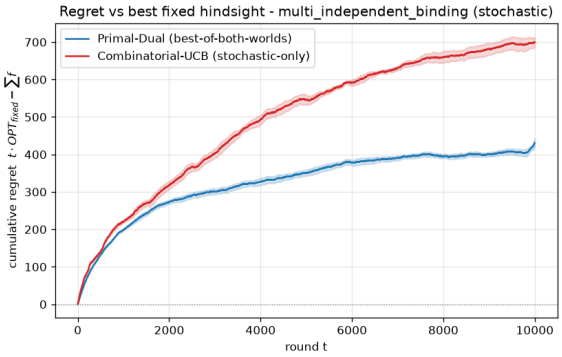


(a) `ns_fast_binding`

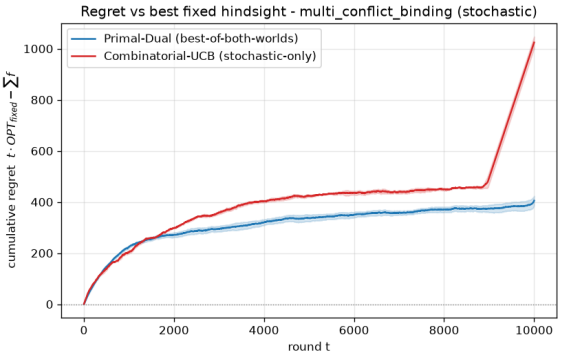


(b) `ns_conflict_binding`

Figure 31: Requirement 3 regret plots, part II.



(a) `multi_independent_binding`



(b) `multi_conflict_binding`

Figure 32: Requirement 3 regret plots on reused stochastic scenarios, part I.

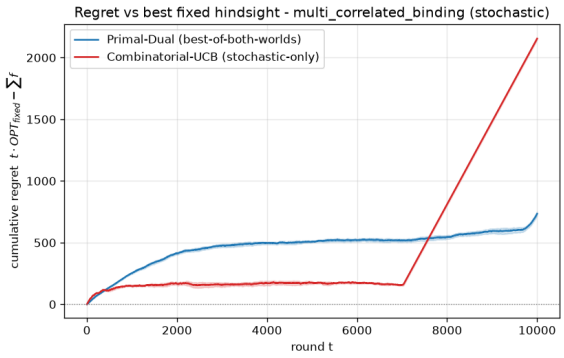
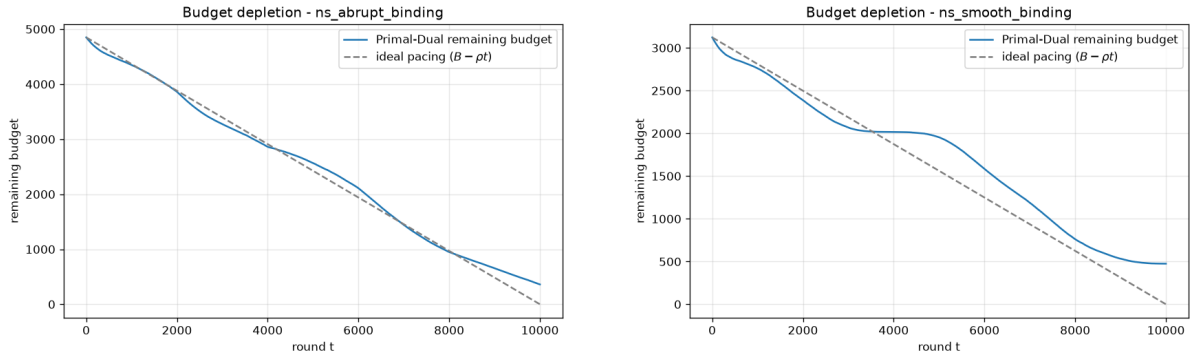
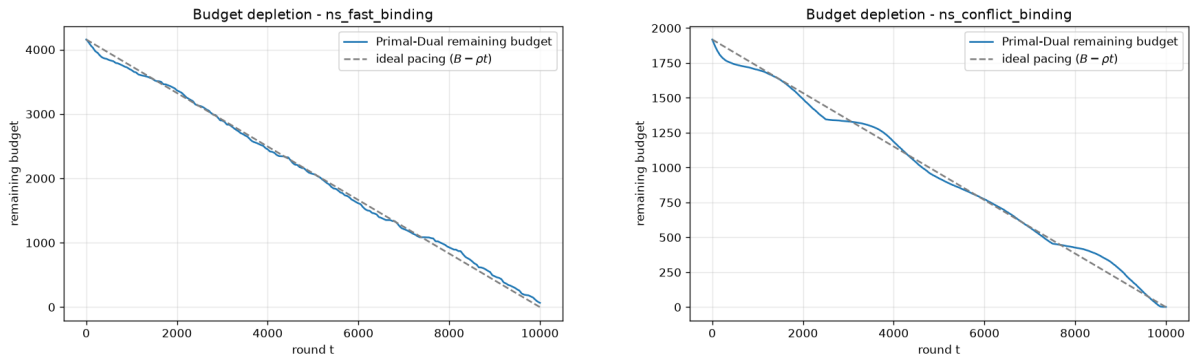


Figure 33: Requirement 3 regret plot on reused stochastic scenario: `multi_correlated_binding`.



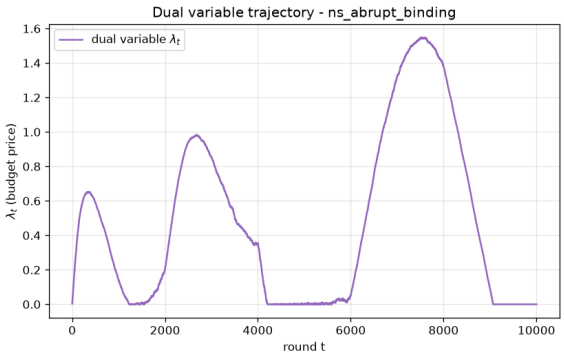
(a) `ns_abrupt_binding` (b) `ns_smooth_binding`

Figure 34: Requirement 3 budget plots, part I.

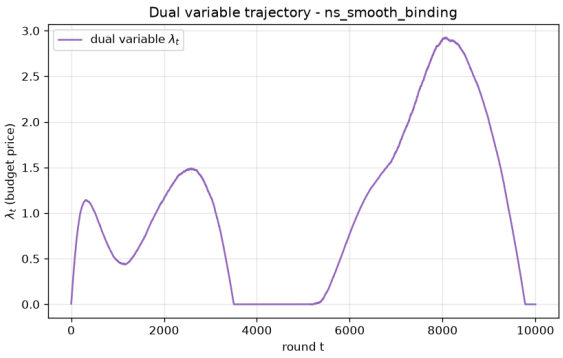


(a) `ns_fast_binding` (b) `ns_conflict_binding`

Figure 35: Requirement 3 budget plots, part II.



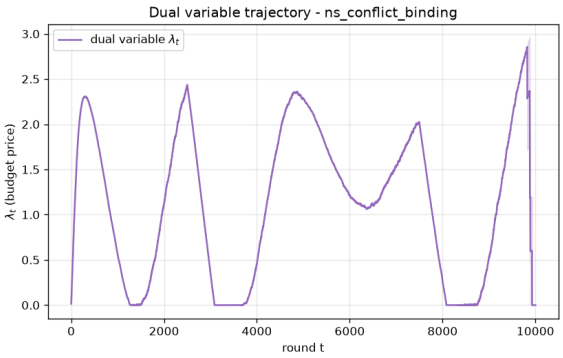
(a) `ns_abrupt_binding`



(b) `ns_smooth_binding`

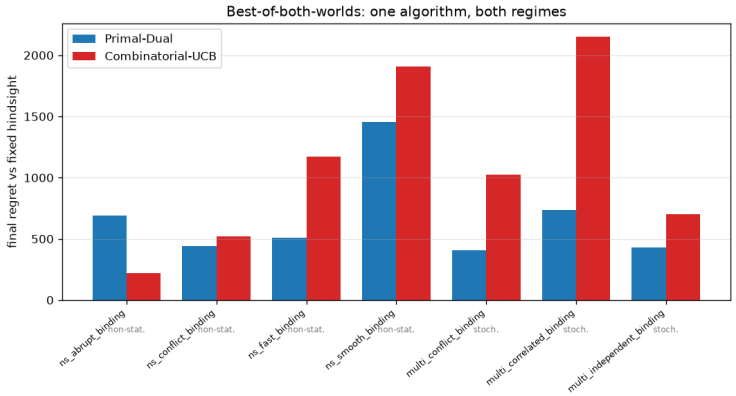
Figure 36: Requirement 3 dual trajectories, part I.

(a) `ns_fast_binding`

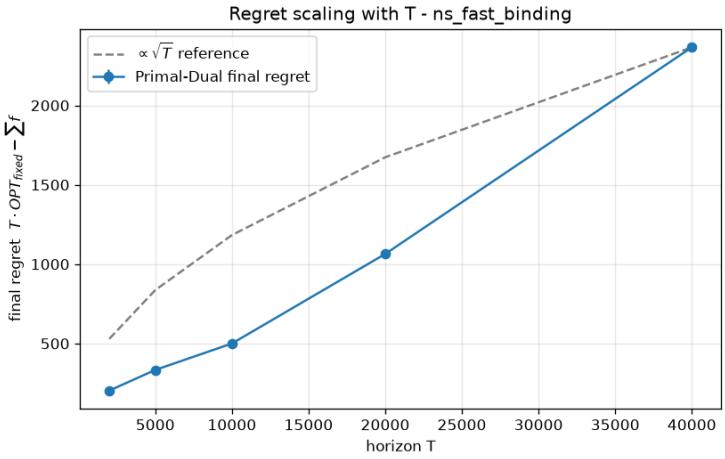


(b) `ns_conflict_binding`

Figure 37: Requirement 3 dual trajectories, part II.

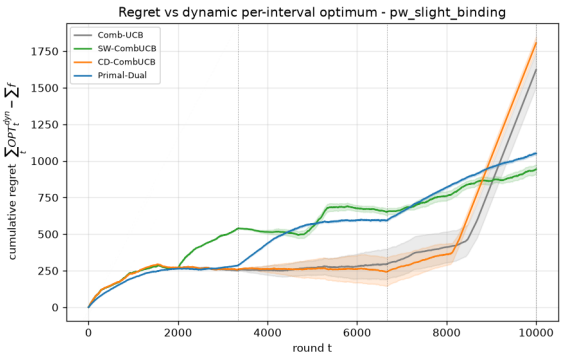


(a) Best-of-both-worlds summary

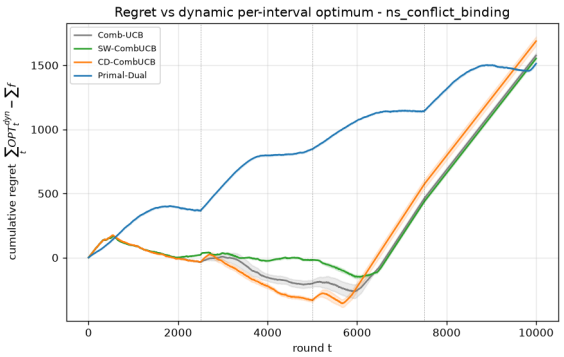


(b) Scaling experiment

Figure 38: Requirement 3 summary figures.

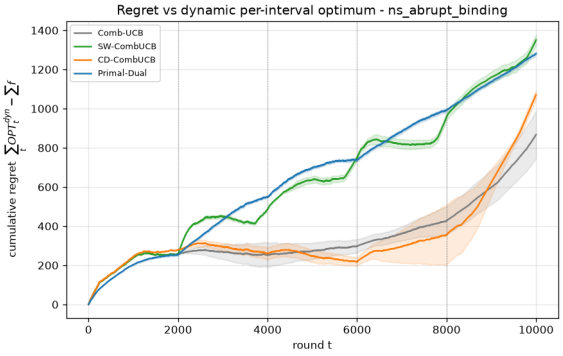


(a) `pw_slight_binding`

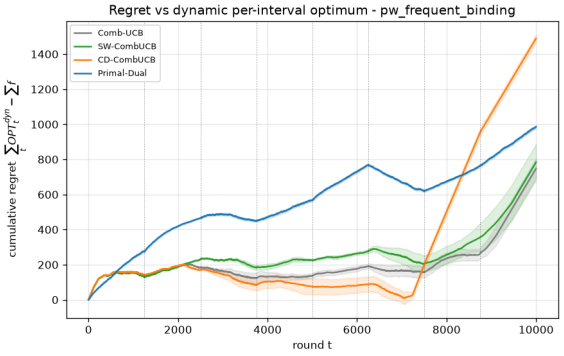


(b) `ns_conflict_binding`

Figure 39: Requirement 4 regret plots, part I.

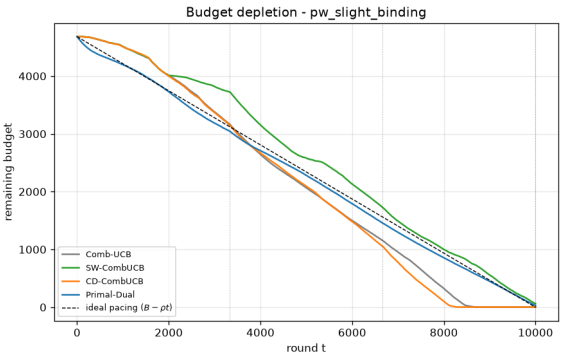


(a) `ns_abrupt_binding`

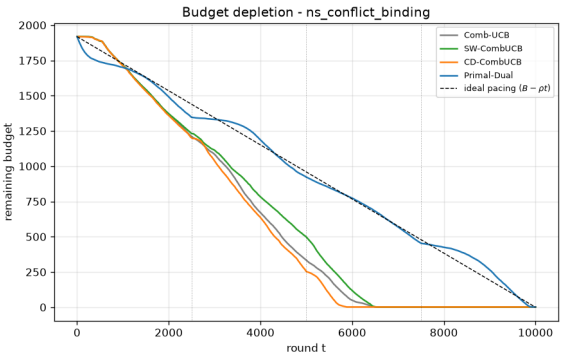


(b) `pw_frequent_binding`

Figure 40: Requirement 4 regret plots, part II.

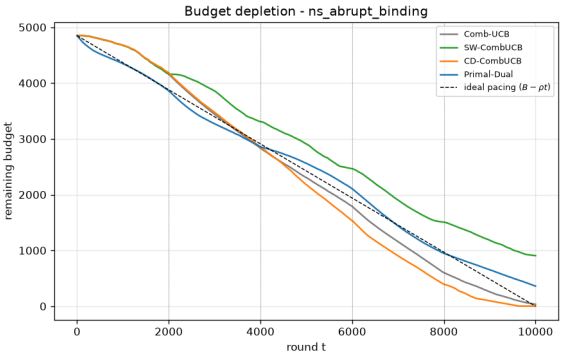


(a) `pw_slight_binding`

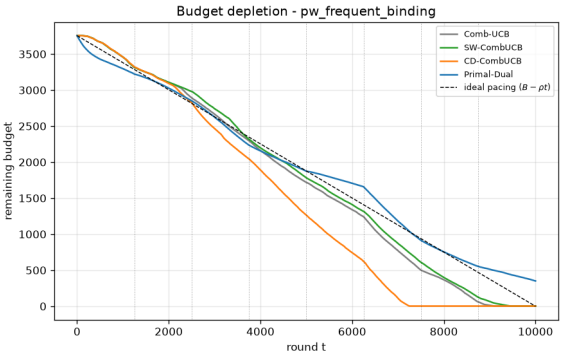


(b) `ns_conflict_binding`

Figure 41: Requirement 4 budget plots, part I.

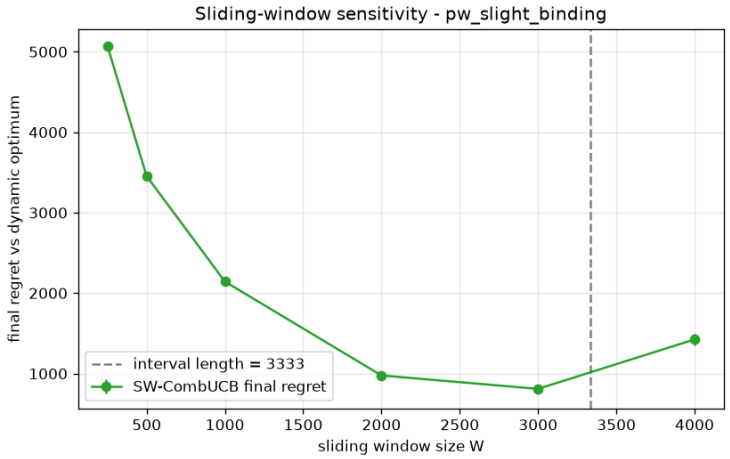


(a) `ns_abrupt_binding`

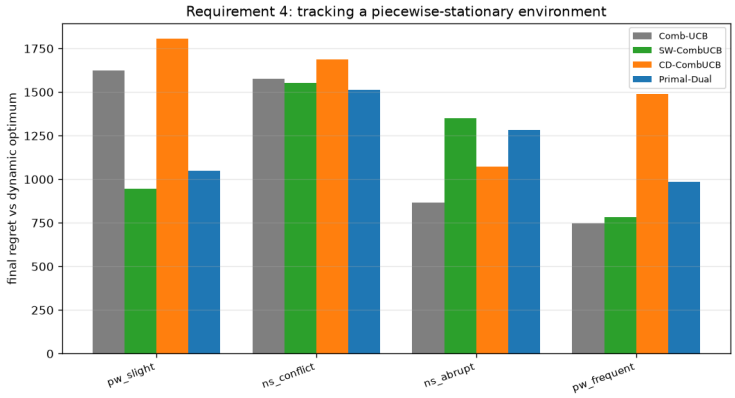


(b) `pw_frequent_binding`

Figure 42: Requirement 4 budget plots, part II.



(a) Window sensitivity



(b) Requirement 4 summary

Figure 43: Requirement 4 summary figures.